Smart Lab & AI Interpretation System – Architecture & Design Document

Version: 1.0.0

Tech Stack: .NET 8/9, EF Core, SQL Server, MediatR (CQRS), Clean / Onion Architecture

1. Executive Summary

Smart Lab & AI Interpretation System is a domain-centric medical laboratory platform that goes beyond data storage to deliver intelligent clinical insights. The system supports lab test booking, result management, OCR extraction from external lab reports, and AI-driven medical recommendations based on patient history and health trends.

2. Project Overview

The goal of the system is to provide a unified platform for managing patients, doctors, and lab orders; capturing and validating lab test results; interpreting those results using a combination of rule-based medical logic and AI services; and tracking health trends over time through patient-centric metrics.

3. Architectural Style and Rationale

The solution adopts Clean Architecture with an Onion-style layering model. Business rules and core medical logic live at the center of the architecture, while infrastructure and frameworks are treated as replaceable implementation details. This separation provides strong boundaries, high testability, and long-term maintainability in a safety-sensitive domain such as medical diagnostics.

This design ensures:

- **Clear separation between core logic and technical tools**
- **Easy testing of medical rules without external dependencies**
- **Long-term maintainability, which is very important in medical systems**

The main reason for choosing this architecture is to keep the system flexible and easy to change in the future.
The core business logic is completely independent from technologies like databases, ASP.NET, or AI services.

This means we can:

- **Change the database or AI provider without affecting the main system**
- **Keep the business rules clean and separated from technical details**

- **Easily add new features or integrations in the future by plugging them in without rewriting existing code**

4. Solution Structure (Layers)

4.1 Domain Layer (Core)

The Domain layer is the core of the system. It contains the main entities (Patient, Doctor, LabTest, TestResult, HealthMetricSnapshot) and domain-specific enumerations (RiskLevel, Gender, OrderStatus, PaymentMethod). It also holds the pure domain logic, including validation rules, age-specific constraints, risk calculation, and trend evaluation. This layer has no dependencies on frameworks or external services.

4.2 Application Layer

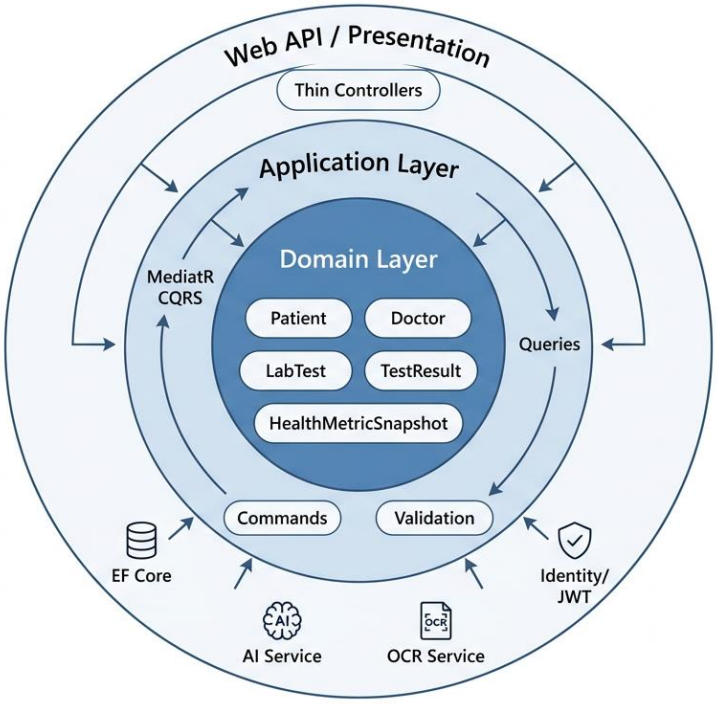
The Application layer orchestrates use cases using MediatR and the CQRS pattern. It defines commands such as BookLabOrder, RegisterTestResult, RunOCRForReport, and InterpretLabResult, and queries such as GetPatientHistory, GetTrendReport, and GetLabOrderDetails. FluentValidation is used to validate incoming requests before the use cases are executed. This layer depends only on the domain model and on abstractions such as repository and service interfaces.

4.3 Infrastructure Layer

The Infrastructure layer provides concrete implementations for the abstractions defined in the inner layers. It includes persistence (EF Core, DbContext, repositories), AI services (implementations of IAIInterpretationService using OpenAI, local ML models, or rule-based engines), OCR services (IOCRService implementations such as Tesseract or Azure Form Recognizer), and Identity and security (ASP.NET Identity, JWT-based authentication).

4.4 Presentation Layer (Web API)

The Presentation layer consists of thin ASP.NET Core Web API controllers. Controllers receive HTTP requests from web or mobile clients, map them to commands or queries, dispatch them through MediatR, and return standardized responses or problem details. No domain logic or data access code is implemented in this layer, which keeps thesurface clean and easy to maintain



5. **Intelligent Interpretation Module**

The intelligent interpretation module is the competitive advantage of the system. When a new TestResult is stored, the system loads the corresponding patient profile and historical HealthMetricSnapshot data. A dedicated Interpretation Service then applies domain rules that take into account test type, patient age, and gender to determine a RiskLevel (for example, High, Medium, or Normal).

If AI interpretation is enabled, the Application layer calls an abstraction such as IAIInterpretationService rather than invoking a concrete AI provider directly. Implementations for this interface may use external APIs (for example, OpenAI), local AI models, or purely rule-based engines. The module merges domain rule outputs, historical trends, and AI insights into a final clinical recommendation, such as "There is a consistent increase in fasting blood glucose over the last three months; an endocrinology consultation is recommended."

6. **AI and OCR Separation Principle**

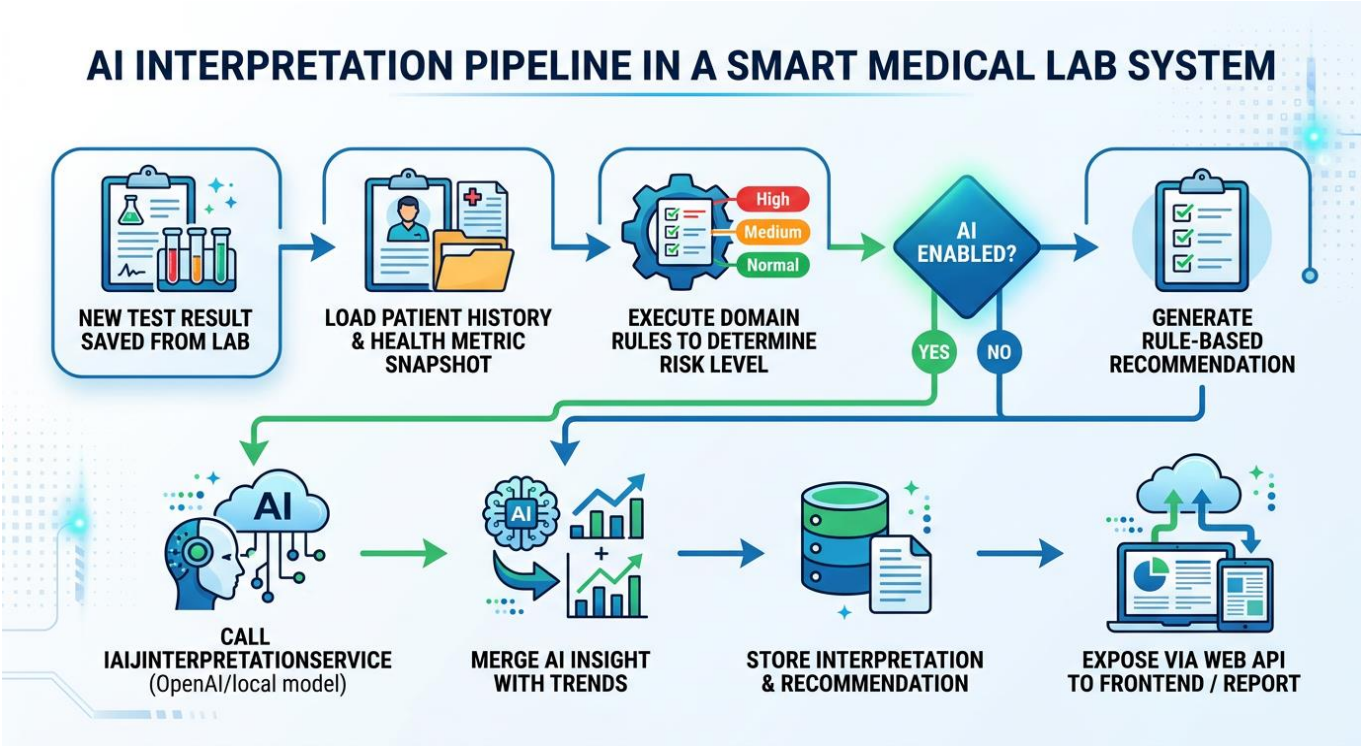
To keep the architecture clean and flexible, the system never calls an AI provider or OCR engine directly from controllers or use case handlers. Instead, it relies on interfaces such as IAIInterpretationService and IOCRService. The Infrastructure layer provides different implementations of these interfaces (for example, using OpenAI, a local Python model, Tesseract OCR, or Azure Form Recognizer). This design allows the team to replace or extend AI and OCR providers without changing the core business logic.

7. High-Level Architecture Diagram

Figure 1 illustrates the Clean / Onion Architecture for the Smart Lab & AI Interpretation System. The Domain layer is at the center, surrounded by the Application layer, the Infrastructure layer, and the outer Web API / Presentation layer.

[Insert Architecture Diagram Image Here – lab_clean_arch.png]

8. AI Interpretation Flowchart

The following flow describes how a raw lab test result is transformed into an interpreted recommendation:

1. A new TestResult is entered and saved to the database.
2. The system loads the Patient profile and historical HealthMetricSnapshot data.
3. Domain rules compute relevant health trends and determine an initial RiskLevel.
4. A decision is made: is AI interpretation enabled for this test or tenant?
- If not enabled, the system generates a purely rule-based recommendation.
- If enabled, the system calls IAIInterpretationService to obtain an AI-generated summary and insight.
5. The outputs of domain rules, trends, and AI insight are merged into a final interpretation.
6. The interpretation and recommendation are stored and exposed via the Web API and reports.